Лабораторная работа №19

Тема работы: «Разработка и отладка полноэкранных приложений разветвляющихся алгоритмов и программ с использованием основных элементов управления диалоговым окном»

1 Цель работы

Научить разрабатывать, конструировать, испытывать и отлаживать разветвляющиеся алгоритмы и программы с использованием основных элементов управления

2 Задание

2.1. Разработать калькулятор аналогичный калькулятору, установленному на вашем используемом компьютере. Управление калькулятором осуществляется по нажатию мыши.

Рисунок 1 - Образец разработанного приложения «Калькулятор»

2.2. Организовать проверку корректности работы программы:

− обработка исключения деления на «0»;

− обработка ввода некорректных данных: ввод букв/символов/знаков;

− сохранение результата для выполнения следующего действия;

− обработка исключения извлечения квадратного корня из отрицательного числа.

3 Оснащение

ПК, задание по варианту, среда разработки Visual Studio.

4 Порядок выполнения работы

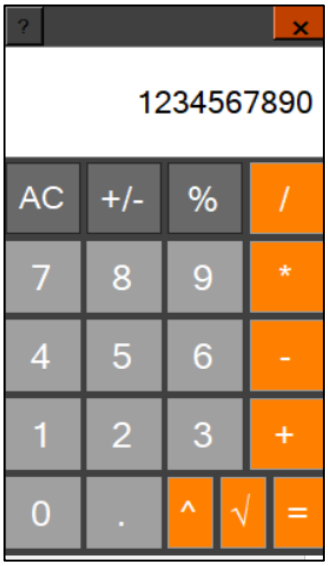

Рисунок 2 – Создание калькулятора

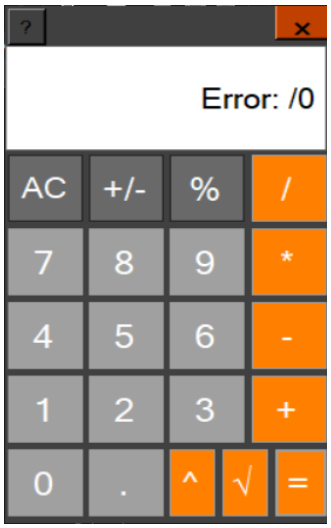

Рисунок 3 – Обработка деления на ноль

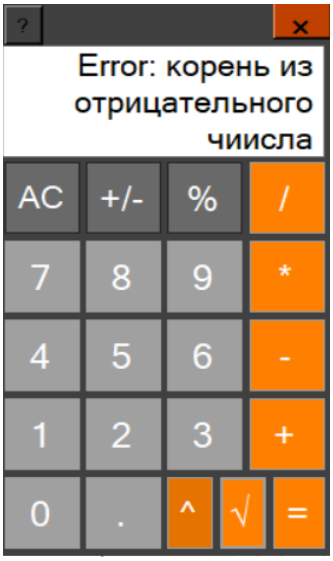

Рисунок 4 – Обработка извлечения корня из отрицательного числа

Код программы:

```
1.      private: double SafeConvert(String^ text) {
2.                      try {
3.
4.                              text = text->Trim();
5.
6.
7.                              if (text == "" || text == "-" || text == "-." || text == "," || text
== "-,") {
8.                                      return 0;
9.                              }
10.
11.                             text = text->Replace(",", ".");
12.
13.
14.                             if (text == "." || text == "-.") {
15.                                     return 0;
16.                             }
17.
18.
19.                             return Double::Parse(text,
20.                                     System::Globalization::CultureInfo::InvariantCulture);
21.                     }
22.                     catch (Exception^) {
23.
24.                             return 0;
25.                     }
26.             }
27. private: System::Void MyForm_Load(System::Object^ sender, System::EventArgs^ e) {
28.     firstOperand = 0;
29.     currentOperation = "";
30.     isNewOperation = true;
31.     hasError = false;
32.     this->label1->Text = "";
33.
34.     this->label1->AutoSize = false;
35.
36.
37.     this->label1->Width = this->ClientSize.Width - 5;
38.     this->label1->Height = 80;
39.     this->label1->Font = gcnew System::Drawing::Font("Microsoft Sans Serif", 16);
40.
41.     this->label1->TextAlign = ContentAlignment::MiddleRight;
42.
43.     this->label1->Text = "";
44.     this->label1->BackColor = Color::White;
45.     this->label1->BorderStyle = Windows::Forms::BorderStyle::FixedSingle;
46.
47.     this->label1->Location = System::Drawing::Point(0, 30);
48. }
49.
50. private: System::Void button10_Click(System::Object^ sender, System::EventArgs^ e) {
51.     if (this->label1->Text == "Error") {
52.             this->label1->Text = "0";
53.             isNewOperation = false;
54.             hasError = false;
55.     }
56.
57.     else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
58.             this->label1->Text = "0";
59.             isNewOperation = false;
60.     }
61.
62.     else {
63.             this->label1->Text += "0";
64.     }
65. }
66.
67.
68. private: System::Void button7_Click(System::Object^ sender, System::EventArgs^ e) {
69.
70.     if (this->label1->Text == "Error") {
71.             this->label1->Text = "1";
72.             isNewOperation = false;
73.             hasError = false;
```

```
74.      }
75.
76.      else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
77.                this->label1->Text = "1";
78.                isNewOperation = false;
79.      }
80.      else {
81.                this->label1->Text += "1";
82.      }
83. }
84.
85.
86. private: System::Void button8_Click(System::Object^ sender, System::EventArgs^ e) {
87.      if (this->label1->Text == "Error") {
88.                this->label1->Text = "2";
89.                isNewOperation = false;
90.                hasError = false;
91.      }
92.      else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
93.                this->label1->Text = "2";
94.                isNewOperation = false;
95.      }
96.      else {
97.                this->label1->Text += "2";
98.      }
99. }
100.
101.
102. private: System::Void button9_Click(System::Object^ sender, System::EventArgs^ e) {
103.      if (this->label1->Text == "Error") {
104.                this->label1->Text = "3";
105.                isNewOperation = false;
106.                hasError = false;
107.      }
108.      else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
109.                this->label1->Text = "3";
110.                isNewOperation = false;
111.      }
112.      else {
113.                this->label1->Text += "3";
114.      }
115. }
116.
117.
118. private: System::Void button4_Click(System::Object^ sender, System::EventArgs^ e) {
119.      if (this->label1->Text == "Error") {
120.                this->label1->Text = "4";
121.                isNewOperation = false;
122.                hasError = false;
123.      }
124.      else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
125.                this->label1->Text = "4";
126.                isNewOperation = false;
127.      }
128.      else {
129.                this->label1->Text += "4";
130.      }
131. }
132.
133.
134. private: System::Void button5_Click(System::Object^ sender, System::EventArgs^ e) {
135.      if (this->label1->Text == "Error") {
136.                this->label1->Text = "5";
137.                isNewOperation = false;
138.                hasError = false;
139.      }
140.      else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
141.                this->label1->Text = "5";
142.                isNewOperation = false;
143.      }
144.      else {
145.                this->label1->Text += "5";
146.      }
147. }
148.
149.
```

```
private: System::Void button6_Click(System::Object^ sender, System::EventArgs^ e) {
    if (this->label1->Text == "Error") {
            this->label1->Text = "6";
            isNewOperation = false;
            hasError = false;
    }
    else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
            this->label1->Text = "6";
            isNewOperation = false;
    }
    else {
            this->label1->Text += "6";
    }
}


private: System::Void button1_Click(System::Object^ sender, System::EventArgs^ e) {
    if (this->label1->Text == "Error") {
            this->label1->Text = "7";
            isNewOperation = false;
            hasError = false;
    }
    else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
            this->label1->Text = "7";
            isNewOperation = false;
    }
    else {
            this->label1->Text += "7";
    }
}

private: System::Void button2_Click(System::Object^ sender, System::EventArgs^ e) {
    if (this->label1->Text == "Error") {
            this->label1->Text = "8";
            isNewOperation = false;
            hasError = false;
    }
    else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
            this->label1->Text = "8";
            isNewOperation = false;
    }
    else {
            this->label1->Text += "8";
    }
}


private: System::Void button3_Click(System::Object^ sender, System::EventArgs^ e) {
    if (this->label1->Text == "Error") {
            this->label1->Text = "9";
            isNewOperation = false;
            hasError = false;
    }
    else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
            this->label1->Text = "9";
            isNewOperation = false;
    }
    else {
            this->label1->Text += "9";
    }
}

private: System::Void button11_Click(System::Object^ sender, System::EventArgs^ e) {

    if (this->label1->Text == "Error") {
            this->label1->Text = "0,";
            isNewOperation = false;
            hasError = false;
            return;
    }


    String^ currentText = this->label1->Text;

    if (isNewOperation || currentText == "" || currentText == "-") {
            this->label1->Text = "0,";
```

```
                isNewOperation = false;
                return;
        }

        else if (currentText == "0" || currentText == "-0") {
                this->label1->Text = currentText + ",";
                isNewOperation = false;
                return;
        }

        bool hasDecimalSeparator = currentText->Contains(",") || currentText->Contains(".");


        if (!hasDecimalSeparator) {
                this->label1->Text = currentText + ",";
        }

}




private: System::Void button23_Click(System::Object^ sender, System::EventArgs^ e) {
        this->Close();
}
private: System::Void button12_Click(System::Object^ sender, System::EventArgs^ e) {
        this->label1->Text = "0";
        firstOperand = 0;
        currentOperation = "";
        isNewOperation = true;
        hasError = false;
}
private: System::Void button13_Click(System::Object^ sender, System::EventArgs^ e) {
        try {
                String^ currentText = this->label1->Text;

                if (currentText == "" || currentText == "0") {
                        this->label1->Text = "-0";
                        isNewOperation = false;
                }

                else if (currentText->StartsWith("-")) {

                        if (currentText == "-0") {
                                this->label1->Text = "0";
                        }
                        else {
                                this->label1->Text = currentText->Substring(1);
                        }
                }
                else {
                        this->label1->Text = "-" + currentText;
                }
        }
        catch (Exception^ ex) {
                this->label1->Text = "Error";
        }
}
private: System::Void button14_Click(System::Object^ sender, System::EventArgs^ e) {
        try {
                String^ currentText = this->label1->Text;


                if (currentText == "" || currentText == "0") {
                        return;
                }


                currentText = currentText->Trim();


                double value = System::Convert::ToDouble(currentText);


                value = value / 100.0;

```

```
302.
303.                if (Math::Abs(value) < 0.000001 && value != 0) {
304.
305.                          this->label1->Text = value.ToString("E6");
306.                }
307.                else if (Math::Abs(value) < 1.0) {
308.
309.                          this->label1->Text = value.ToString("0.##########");
310.                }
311.                else {
312.
313.                          this->label1->Text = value.ToString();
314.                }
315.
316.                isNewOperation = true;
317.      }
318.      catch (FormatException^) {
319.                this->label1->Text = "Error: не число";
320.      }
321.      catch (OverflowException^) {
322.                this->label1->Text = "Error: слишком большое";
323.      }
324.      catch (Exception^) {
325.                this->label1->Text = "Error";
326.      }
327. }
328.
329.
330. private: System::Void button18_Click(System::Object^ sender, System::EventArgs^ e) {
331.      if (this->label1->Text != "" && this->label1->Text != "Error") {
332.                firstOperand = SafeConvert(this->label1->Text);
333.                currentOperation = "+";
334.                isNewOperation = true;
335.      }
336. }
337.
338.
339. private: System::Void button17_Click(System::Object^ sender, System::EventArgs^ e) {
340.      if (this->label1->Text != "" && this->label1->Text != "Error") {
341.                firstOperand = SafeConvert(this->label1->Text);
342.                currentOperation = "-";
343.                isNewOperation = true;
344.      }
345. }
346.
347.
348. private: System::Void button16_Click(System::Object^ sender, System::EventArgs^ e) {
349.      if (this->label1->Text != "" && this->label1->Text != "Error") {
350.                firstOperand = SafeConvert(this->label1->Text);
351.                currentOperation = "*";
352.                isNewOperation = true;
353.      }
354. }
355.
356. private: System::Void button15_Click(System::Object^ sender, System::EventArgs^ e) {
357.      if (this->label1->Text != "" && this->label1->Text != "Error") {
358.                firstOperand = SafeConvert(this->label1->Text);
359.                currentOperation = "/";
360.                isNewOperation = true;
361.      }
362. }
363.
364.
365. private: System::Void button19_Click(System::Object^ sender, System::EventArgs^ e) {
366.      if (this->label1->Text != "" && this->label1->Text != "Error") {
367.                try {
368.                          firstOperand = ParseMyNumber(this->label1->Text);
369.                          currentOperation = "^";
370.                          isNewOperation = true;
371.                }
372.                catch (Exception^) {
373.                          this->label1->Text = "Error";
374.                }
375.      }
376. }
377.
```

```
private: System::Void button21_Click(System::Object^ sender, System::EventArgs^ e) {
    if (currentOperation != "" && this->label1->Text != "" && this->label1->Text != "Error") {
        try {
            double secondOperand = SafeConvert(this->label1->Text);
            double result = 0;

            if (currentOperation == "+") {
                result = firstOperand + secondOperand;
            }
            else if (currentOperation == "-") {
                result = firstOperand - secondOperand;
            }
            else if (currentOperation == "*") {
                result = firstOperand * secondOperand;
            }
            else if (currentOperation == "/") {
                if (secondOperand == 0) {
                    this->label1->Text = "Error: /0";
                    currentOperation = "";
                    isNewOperation = true;
                    return;
                }
                result = firstOperand / secondOperand;
            }
            else if (currentOperation == "^") {
                try {
                    result = Math::Pow(firstOperand, secondOperand);

                    if (Double::IsInfinity(result)) {
                        this->label1->Text = "Error: слишком большое";
                        currentOperation = "";
                        isNewOperation = true;
                        return;
                    }
                    if (Double::IsNaN(result)) {
                        this->label1->Text = "Error";
                        currentOperation = "";
                        isNewOperation = true;
                        return;
                    }
                }
                catch (...) {
                    this->label1->Text = "Error";
                    currentOperation = "";
                    isNewOperation = true;
                    return;
                }
            }


            this->label1->Text = result.ToString();
            currentOperation = "";
            isNewOperation = true;

        }
        catch (...) {
            this->label1->Text = "Error";
            currentOperation = "";
            isNewOperation = true;
        }
    }
}
    private: double ParseMyNumber(String^ text) {
        text = text->Trim();

        if (text == "") return 0;


        if (text == "-") return 0;


        bool hasComma = text->Contains(",");
        bool hasDot = text->Contains(".");

        if (hasComma && hasDot) {
```

```
454.                            throw gcnew FormatException("Неверный формат числа");
455.                    }
456.
457.
458.                    if (hasComma) {
459.                            text = text->Replace(",", ".");
460.                    }
461.
462.
463.                    for (int i = 0; i < text->Length; i++) {
464.                            Char c = text[i];
465.                            if (!(Char::IsDigit(c) || c == '.' || c == '-' || (c == 'E' || c == 'e'))) {
466.                                    throw gcnew FormatException("Недопустимый символ: " + c);
467.                            }
468.                            if (c == '-' && i != 0) {
469.                                    throw gcnew FormatException("Минус не в начале");
470.                            }
471.                    }
472.
473.
474.                    return Double::Parse(text,
475.                            System::Globalization::CultureInfo::InvariantCulture);
476.            }
477.
478.
479. private: System::Void button20_Click(System::Object^ sender, System::EventArgs^ e) {
480.     try {
481.                String^ currentText = this->label1->Text;
482.
483.
484.                if (currentText == "" || currentText == "Error") {
485.                        return;
486.                }
487.
488.                currentText = currentText->Replace(",", ".");
489.                double value = Double::Parse(currentText,
490.                        System::Globalization::CultureInfo::InvariantCulture);
491.
492.
493.                if (value < 0) {
494.                        this->label1->Text = "Error: корень из отрицательного чиисла";
495.                        return;
496.                }
497.
498.
499.                double result = Math::Sqrt(value);
500.
501.
502.                if (Math::Abs(result - Math::Round(result)) < 0.0000001) {
503.
504.                        this->label1->Text = Math::Round(result).ToString();
505.                }
506.                else {
507.                        this->label1->Text = result.ToString("0.##########");
508.                }
509.
510.     }
511.     catch (FormatException^) {
512.                this->label1->Text = "Error: не число";
513.     }
514.     catch (Exception^) {
515.                this->label1->Text = "Error";
516.     }
517. }
518.
519. private: System::Void button22_Click(System::Object^ sender, System::EventArgs^ e) {
520.     this->label1->Text = (this->label1->Text == "Оконное приложение \"Калькулятор\"") ? "0" : "Оконное
приложение \"Калькулятор\"";
521. }
522. };
523. }
524.
```

5 Контрольные вопросы и задания:

1. Дайте краткое понятие разветвляющихся алгоритмов и их применение в программировании на языке C++.

Разветвляющиеся алгоритмы — это алгоритмы, в которых в зависимости от выполнения или невыполнения некоторого условия выполняется одна или другая последовательность действий (ветвь). В программировании на C++ они реализуются с помощью условных операторов (if, else, switch) и позволяют программе принимать решения, делая её поведение гибким и адаптивным к разным входным данным и состояниям. Применяются повсеместно: от проверки корректности ввода и обработки пользовательских команд до реализации сложной бизнес-логики и управления потоком выполнения.

2. Раскройте значение элементов управления в диалоговых окнах и их роль в реализации пользовательского интерфейса.

Элементы управления (виджеты) в диалоговых окнах — это графические компоненты (кнопки, поля ввода, списки, флажки, переключатели и т.д.), которые служат интерфейсом для взаимодействия пользователя с программой. Их роль заключается в:

- Вводе данных: Поля ввода, числовые счётчики.
- Выборе вариантов: Радиокнопки, выпадающие списки, флажки.
- Инициировании действий: Кнопки (ОК, Отмена, Применить).
- Отображении информации: Метки (статические тексты), поля списков.

Они структурируют информацию, направляют пользователя, обеспечивают удобный и интуитивно понятный способ передачи команд и данных программе, являясь фундаментом графического пользовательского интерфейса (GUI).

3. Сформулируйте сущность конструкций ветвления (if, else, switch) и их использование в разработке программ на C++.

Конструкции ветвления — это управляющие операторы языка C++, которые изменяют порядок выполнения программы в зависимости от истинности логических условий.

- if, else: Оператор if проверяет условие в скобках; если оно истинно (true), выполняется следующий за ним блок кода. С помощью else можно задать альтернативный блок для случая, когда условие ложно (false). Цепочка else if позволяет проверять несколько условий последовательно.
- switch: Этот оператор обеспечивает многовариантное ветвление на основе сравнения целочисленного или перечисляемого (enum) выражения с набором константных значений (case). Он часто используется как более читаемая альтернатива длинным цепочкам if-else if при выборе одного из многих вариантов.

Их использование позволяет создавать нелинейные программы, которые реагируют на различные события, данные пользователя и внутренние состояния системы.

4. Изложите особенности разработки полноэкранных приложений с использованием разветвляющихся алгоритмов на языке C++.

При разработке полноэкранных приложений (например, игр, графических редакторов, систем моделирования) разветвляющиеся алгоритмы играют ключевую роль в управлении сложным интерактивным поведением. Особенности включают:

– Управление состоянием приложения: Алгоритмы ветвления используются для переключения между состояниями (меню, игра, пауза, настройки) на основе событий (нажатие клавиши, выбор в меню).

– Обработка взаимодействия: Определение, с каким объектом на экране взаимодействует пользователь (клик мыши, касание), и выбор соответствующей реакции.

– Игровая и бизнес-логика: Реализация искусственного интеллекта (выбор поведения противника), проверка условий победы/поражения, обработка коллизий объектов.

– Адаптация интерфейса: Изменение видимых элементов интерфейса в зависимости от контекста (например, скрытие панелей инструментов в определенном режиме).

Разветвления становятся центральным механизмом для реагирования на непрерывный поток событий в реальном времени.

5. Раскройте особенности отладки программ с разветвляющимися алгоритмами и какие методы можно использовать для выявления ошибок.

Особенность отладки разветвляющихся алгоритмов заключается в необходимости проверять корректность работы всех возможных путей выполнения (ветвей) программы, что усложняется с ростом количества условий.

Методы для выявления ошибок:

1. Пошаговое выполнение (Step Over/Into): Позволяет отследить, по какой именно ветви пошло выполнение, и проверить корректность вычисления условий.

2. Установка контрольных точек (Breakpoints): Особенно полезно ставить точки на условиях (if, switch) и внутри различных ветвей.

3. Просмотр и проверка значений переменных (Watch): Критически важно контролировать переменные, используемые в условиях, чтобы убедиться, что они содержат ожидаемые значения.

4. Ведение журнала (Logging): Вывод в консоль или файл значений переменных и сообщений о входе в определенные ветви (например, "Условие A выполнено", "Выбран case 3").

5. Исчерпывающее тестирование: Создание набора тестовых данных, которые гарантированно покрывают все возможные условия (положительные/отрицательные числа, граничные значения, все варианты case).

6. Статический анализ кода: Использование средств IDE или внешних инструментов для поиска потенциально недостижимого кода (dead code) или логических противоречий.

6. Дайте определение событиям и обработчикам событий в контексте работы с диалоговыми окнами на C++.

В контексте диалоговых окон на C++ (при использовании фреймворков, таких как Qt, MFC, wxWidgets):

– Событие (Event) — это сообщение, генерируемое системой или библиотекой в ответ на действие пользователя (нажатие кнопки мыши, ввод текста, выбор элемента списка) или на системное происшествие (таймер, перерисовка окна).

– Обработчик события (Event Handler) — это функция или метод-член класса, который вызывается автоматически фреймворком при наступлении конкретного события, связанного с определенным элементом управления. Его задача — выполнить код, который должен отреагировать на событие (например, при нажатии кнопки "Рассчитать" прочитать данные из полей ввода, выполнить вычисления и вывести результат).

Программист связывает (коннектит) события интерфейса с соответствующими обработчиками, реализуя тем самым реакцию программы на действия пользователя.

7. Сформулируйте особенности использования графических библиотек для создания полноэкранных приложений с диалоговыми окнами на C++.

Графические библиотеки (фреймворки) такие как Qt, SFML, SDL (часто в связке с библиотеками для GUI), Unreal Engine, предоставляют абстракцию над низкоуровневыми системными вызовами, упрощая разработку. Особенности их использования:

– Цикл обработки событий (Event Loop): Библиотека предоставляет центральный цикл, который постоянно опрашивает систему на наличие событий (ввод, таймеры) и распределяет их по соответствующим виджетам и объектам.

– Иерархия виджетов/объектов: Элементы интерфейса (окна, кнопки) являются объектами классов библиотеки, что позволяет управлять ими через свойства и методы, а также строить древовидную структуру.

– Сигналы и слоты (в Qt) или другие системы callback'ов: Это механизмы для гибкой и безопасной связи событий (сигналов) с функциями-обработчиками (слотами), что является основой реактивного программирования.

– Инструменты для быстрого прототипирования: Многие библиотеки имеют визуальные редакторы интерфейса (Qt Designer), позволяющие "рисовать" диалоговые окна, а затем наполнять их логикой на C++.

– Абстракция от платформы: Код, написанный с использованием таких библиотек, часто может быть скомпилирован для разных операционных систем без значительных изменений.

– Совмещение графики и GUI: Библиотеки позволяют интегрировать сложную графику (рендеринг 2D/3D, анимацию) в полноэкранном режиме с наложенными поверх диалоговыми окнами и элементами управления для создания комплексных интерактивных приложений.